Komikan Concurrente

Informe de diseño y sincronización

Tarea 2 Sistemas Operativos

Integrantes: Lucas Herrada y Joakin Mac-Auliffe

1. Diseño del esquema de sincronización

Arquitectura: todo con hilos en un solo proceso

Usamos hilos POSIX y no procesos porque el tablero queda naturalmente en memoria compartida y no hace falta `shm_open` ni `mmap`, que son justo los recursos que quedan huérfanos si algo falla. Por la misma razón los semáforos son sin nombre y se crean con `sem_init`, así que no dejan archivos en `/dev/shm`. Los hilos son: `main`, un matchmaker, N hilos de jugador, un juez por partida con a lo más M a la vez y 13 hilos de ficha por partida, que son un puma y 12 perros.

1. Cada jugador llega en un momento aleatorio, se encola en una cola FIFO protegida por mutex y espera sin consumir CPU en su propio semáforo.
2. El matchmaker saca a dos jugadores, donde el primero es el Puma, y hace `sem_wait` sobre un semáforo iniciado en M. Si las M mesas están ocupadas, la pareja espera ahí. Si N es impar, el último jugador se retira sin jugar.
3. Las M mesas son un pool fijo de estructuras reutilizables, así que no hay `malloc` por partida. El matchmaker lanza un juez con la mesa elegida.
4. El juez inicializa el tablero y el mutex de la mesa y recién ahí avisa a los dos jugadores. Cada jugador crea y espera sus propias fichas, y el juez solo arbitra.
5. Al terminar la partida, el juez espera a que los dos jugadores dejen de usar la mesa mediante un contador atómico, la destruye y la devuelve al pool con `sem_post`.

Exclusión mutua: un mutex por mesa

Todo movimiento, que incluye elegir destino, mover o capturar, escribir el log y revisar si alguien ganó, se hace completo dentro de `mesa_mover_perro` o `mesa_mover_puma`, con el mutex de la mesa tomado. Si dos fichas quieren la misma casilla, la que toma el mutex primero se queda con ella y la otra la ve ocupada y elige otra. Los retardos aleatorios entre movimientos, de 50 a 300 ms por defecto, van siempre fuera de la sección crítica. Los hilos de ficha nunca tocan el mutex, así que toda la sincronización está en `mesa.c` y no se puede olvidar un `unlock` en ninguna rama. Tres detalles importan para la corrección:

- Cada ficha tiene un id, 0 para el puma y 1 a 12 para los perros, y el tablero guarda su posición. Al capturar, el puma marca al perro como capturado y su hilo termina. Si solo se marcara la casilla, otro perro podría ocupar esa casilla y el hilo del capturado creería seguir vivo.
- `partida_activa` es un `atomic_int`: se lee sin mutex en el ciclo de cada hilo y se vuelve a revisar dentro del mutex antes de mover, así nadie mueve después de que alguien ganó. Con `volatile` ThreadSanitizer marcaba una carrera.
- El salto del puma usa una tabla precalculada de 70 saltos de tres nodos en línea recta, incluido el triángulo, en vez de calcular geometría en cada movimiento.

Descartamos una cola de peticiones procesada por el juez: sumaba otra estructura de sincronización y un cuello de botella sin ganar corrección. Un mutex por casilla daría más concurrencia, pero el salto del puma necesita tres casillas a la vez y obligaría a ordenar los locks para evitar deadlock.

2. Análisis de riesgos de concurrencia

Deadlock. Con un solo mutex por tablero no hay espera circular entre casillas. El único otro mutex que se toma dentro de la sección crítica es el del log, que es global para que dos mesas no mezclen líneas, y siempre en el orden mesa y después log. Nunca se hace `sem_wait` con el mutex de una mesa tomado, y la pareja espera su mesa sin retener ningún recurso. Un error real: el juez destruía la mesa apenas bajaba la bandera y fichas aún activas usaban un mutex destruido, cosa que ThreadSanitizer marcó, y se corrigió esperando a los dos jugadores antes de destruir.

Livelock. Una partida podría no terminar nunca si el puma esquiva y los perros deambulan al azar. Cada mesa tiene un tope de movimientos, por defecto 1000 y configurable con `max_movimientos`, y al llegar a él la partida queda en empate.

Starvation. El mutex de pthread no es FIFO, así que ninguna ficha tiene garantía de turno. Lo aceptamos porque los retardos aleatorios y una sección crítica de microsegundos hacen la contención despreciable, como se ve en la evaluación. Los jugadores se emparejan en orden de llegada, y como solo el matchmaker espera por las mesas no hay competencia entre parejas.

Abandono. Nunca usamos `pthread_cancel`. Para probar el fallo de un jugador de forma reproducible, el `config.txt` tiene las claves `abandono_jugador` y `abandono_ms`: ese jugador deja de mover sus fichas a los `abandono_ms` de empezar su partida, sin avisar a nadie. Sus hilos terminan solos fuera de la sección crítica, así que no queda ningún mutex retenido. El juez lo detecta con latidos: cada intento de movimiento anota la hora de su bando y, si un bando lleva más de un umbral sin latido, el juez declara ganador al oponente. El umbral es 5 veces `delay_max` con un mínimo de 2 s: con un umbral fijo de 5 s y retardos de 6 a 7 s, el juez declaraba abandonos falsos, cosa que comprobamos.

Ctrl+C. El manejador solo baja la bandera atómica `servidor_activo` y usa `write`, que es seguro en señales. El matchmaker espera con `sem_timedwait` de 100 ms, así que la ve rápido, deja de emparejar, cierra la cola y despide a los jugadores que seguían esperando; los que lleguen después también se retiran. Los jueces cancelan las partidas en curso y esperan a sus jugadores antes de liberar la mesa. `main` hace `join` solo de los hilos que alcanzó a crear y libera semáforos, mutex y memoria. Antes, con N=500 y M=2, un Ctrl+C a los 2 s tardaba 8,5 s y arrancaba las 250 partidas.

3. Evaluación

Elegimos el diseño más simple que garantiza corrección: un mutex monolítico por mesa, con la sincronización encapsulada en un solo archivo. El costo de esa simplicidad, que es no tener concurrencia dentro de una mesa, casi no se paga. Medimos con el log escrito a archivo y retardos de 50 a 300 ms, con N=6 y M=1 y una corrida por fila:

Sección crítica, 622 intentos	promedio 3 μs, máximo 48 μs
Intentos que encontraron el mutex ocupado	0 de 622, y 0 de 649 con retardos de 5 a 20 ms
Torneo de 8 jugadores y 4 partidas, M=1	13,2 s, o 54 movimientos/s
Mismo torneo con M=4	4,1 s, o 182 movimientos/s

La sección crítica dura unos cuatro órdenes de magnitud menos que el retardo entre movimientos, que es de 50 ms como mínimo, por eso el mutex único no degrada el rendimiento. Entre mesas la concurrencia es total, porque cada una tiene su propio mutex y solo comparten el del log, y por eso el mismo torneo se acelera al aumentar M, aunque la duración de las partidas es aleatoria y los tiempos varían entre corridas.

Validamos con ThreadSanitizer, que dio 0 reportes en corridas normales, con abandono simulado y con Ctrl+C a mitad de partida, y con AddressSanitizer, UBSan y LeakSanitizer, que dieron 0 errores y 0 fugas. Con N igual a 1, 2, 3, 7, 10 y 40 el programa terminó solo y las partidas simultáneas nunca superaron M. Con N=500 y M=2, un Ctrl+C a los 2 s termina el programa en 0,3 s con solo las 2 partidas en curso iniciadas. Probamos también el abandono simulado, la disposición inicial por archivo y el tope de movimientos. Limitación: como los perros se mueven al azar, el puma ganó todas las partidas aleatorias; la victoria de los perros solo la probamos con una disposición armada a propósito.